Entrega Lab CI/CD - Desafio RH

1. Link do repositório do projeto

https://github.com/anablefari/desafio-rh

2. Arquivo da pipeline

.github/workflows/lab-cicd.yml

Conteúdo integral do arquivo:

2. Arquivo da pipeline - parte 1 de 4

```
name: Lab CI/CD - Desafio RH

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: lab-cicd-${{ github.ref }}
  cancel-in-progress: true

jobs:
  ci-backend:
    name: Backend - testes e imagem Docker
    runs-on: ubuntu-latest
    timeout-minutes: 15
    defaults:
      run:
        working-directory: backend

    steps:
      - name: Checkout do repositório
        uses: actions/checkout@v7

      - name: Configurar Java 21
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
          cache-dependency-path: backend/pom.xml

      - name: Compilar e executar testes
        run: bash ./mvnw --batch-mode --no-transfer-progress clean verify

      - name: Configurar Docker Buildx
        uses: docker/setup-buildx-action@v4

      - name: Validar build da imagem Docker
        uses: docker/build-push-action@v7
        with:
```

2. Arquivo da pipeline - parte 2 de 4

```
046           context: backend
047           file: backend/Dockerfile
048           platforms: linux/amd64
049           push: false
050           cache-from: type=gha,scope=backend
051           cache-to: type=gha,mode=max,scope=backend
052
053   ci-frontend:
054     name: Frontend - lint e build
055     runs-on: ubuntu-latest
056     timeout-minutes: 10
057     defaults:
058       run:
059         working-directory: frontend
060
061     steps:
062       - name: Checkout do repositório
063         uses: actions/checkout@v7
064
065       - name: Configurar Node.js 24
066         uses: actions/setup-node@v7
067         with:
068           node-version: '24'
069           cache: npm
070           cache-dependency-path: frontend/package-lock.json
071
072       - name: Instalar dependências
073         run: npm ci
074
075       - name: Analisar código com ESLint
076         run: npm run lint
077
078       - name: Compilar frontend
079         run: npm run build
080
081   cd-backend:
082     name: CD Backend - Docker Hub e Render
083     needs: [ci-backend, ci-frontend]
084     if: github.event_name == 'push' && github.ref == 'refs/heads/main'
085     runs-on: ubuntu-latest
086     timeout-minutes: 15
087     environment: production-backend
088     env:
089       DOCKER_IMAGE: ${{ vars.DOCKERHUB_IMAGE }}
090       DOCKERHUB_USERNAME: ${{ vars.DOCKERHUB_USERNAME }}
```

2. Arquivo da pipeline - parte 3 de 4

```
091       DOCKERHUB_TOKEN: ${{ secrets.DOCKERHUB_TOKEN }}
092       RENDER_DEPLOY_HOOK_URL: ${{ secrets.RENDER_DEPLOY_HOOK_URL }}
093
094     steps:
095       - name: Checkout do repositório
096         uses: actions/checkout@v7
097
098       - name: Validar configuração do backend
099         shell: bash
100         run: |
101           if [ -z "$DOCKER_IMAGE" ] || [ -z "$DOCKERHUB_USERNAME" ] || [ -z "$DOCKERHUB_TOKEN" ] || [ -z "$RENDER_DEPLOY_HOOK_URL" ]; then
102             echo "Configure DOCKERHUB_IMAGE e DOCKERHUB_USERNAME como Variables; DOCKERHUB_TOKEN e RENDER_DEPLOY_HOOK_URL como Secrets."
103             exit 1
104           fi
105
106       - name: Autenticar no Docker Hub
107         uses: docker/login-action@v4
108         with:
109           username: ${{ env.DOCKERHUB_USERNAME }}
110           password: ${{ env.DOCKERHUB_TOKEN }}
111
112       - name: Configurar Docker Buildx
113         uses: docker/setup-buildx-action@v4
114
115       - name: Publicar imagem do backend
116         uses: docker/build-push-action@v7
117         with:
118           context: backend
119           file: backend/Dockerfile
120           platforms: linux/amd64
121           push: true
122           tags: |
123             ${{ env.DOCKER_IMAGE }}:latest
124             ${{ env.DOCKER_IMAGE }}:${{ github.sha }}
125           cache-from: type=gha,scope=backend
126           cache-to: type=gha,mode=max,scope=backend
127
128       - name: Publicar a nova imagem no Render
129         shell: bash
130         run: |
131           curl --fail --silent --show-error \
132             --request POST \
133             --get \
134             --data-urlencode "imgURL=docker.io/${DOCKER_IMAGE}:${GITHUB_SHA}" \
135             "$RENDER_DEPLOY_HOOK_URL"
```

2. Arquivo da pipeline - parte 4 de 4

```
136
137   cd-frontend:
138     name: CD Frontend - Vercel
139     needs: [ci-backend, ci-frontend]
140     if: github.event_name == 'push' && github.ref == 'refs/heads/main'
141     runs-on: ubuntu-latest
142     timeout-minutes: 15
143     environment: production-frontend
144     env:
145       VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
146       VERCEL_ORG_ID: ${{ vars.VERCEL_ORG_ID }}
147       VERCEL_PROJECT_ID: ${{ vars.VERCEL_PROJECT_ID }}
148       VITE_API_URL: ${{ vars.BACKEND_PUBLIC_URL }}
149
150     steps:
151       - name: Checkout do repositório
152         uses: actions/checkout@v7
153
154       - name: Configurar Node.js 24
155         uses: actions/setup-node@v7
156         with:
157           node-version: '24'
158
159       - name: Validar configuração do frontend
160         shell: bash
161         run: |
162           if [ -z "$VERCEL_TOKEN" ] || [ -z "$VERCEL_ORG_ID" ] || [ -z "$VERCEL_PROJECT_ID" ] || [ -z "$VITE_API_URL" ]; then
163             echo "Configure VERCEL_TOKEN como Secret; VERCEL_ORG_ID, VERCEL_PROJECT_ID e BACKEND_PUBLIC_URL como Variables."
164             exit 1
165           fi
166
167       - name: Instalar Vercel CLI
168         run: npm install --global vercel@latest
169
170       # O projeto Vercel usa Root Directory = frontend. A CLI parte da raiz
171       # para evitar que a configuração resulte no caminho frontend/frontend.
172       - name: Carregar configurações de produção
173         run: vercel pull --yes --environment=production --token="$VERCEL_TOKEN"
174
175       - name: Preparar build de produção
176         run: vercel build --prod --token="$VERCEL_TOKEN"
177
178       - name: Publicar frontend
179         run: vercel deploy --prebuilt --prod --token="$VERCEL_TOKEN"
```

3. Prompts utilizados

Prompt 1

Analise os PDFs 07 - CI - Integração Contínua, 08 - CI - GitAction_Docker e Lab CICD-Pipeline e examine este projeto full-stack, composto por frontend React com TypeScript e backend Spring Boot com Maven. Identifique os requisitos obrigatórios e proponha uma estratégia de CI/CD com GitHub Actions, Docker Hub, Render e Vercel.

Prompt 3

Implemente do zero a pipeline .github/workflows/lab-cicd.yml. Execute CI em push e pull_request para main; configure Java 21, Maven e validação Docker no backend; configure Node.js 24, npm ci, ESLint e build no frontend; e execute CD somente em push na main depois que as duas CIs forem aprovadas. Publique a imagem no Docker Hub com latest e SHA, acione o Render e publique o frontend na Vercel usando apenas Secrets e Variables.

Prompt 4

Revise a nova pipeline quanto a sintaxe, segurança, dependências entre jobs e condições de deployment. Adicione permissões mínimas, timeouts, cache, cancelamento de execuções antigas e validação das configurações. Atualize o README, valide o YAML e envie a implementação para a branch do fork, registrando as limitações locais.

4. Registro das alterações realizadas

Alteração	Registro
Nova pipeline	Criação de .github/workflows/lab-cicd.yml.
CI do backend	Java 21, cache Maven, clean verify e validação Docker linux/amd64.
CI do frontend	Node.js 24, cache npm, npm ci, ESLint e build Vite.
Imagem Docker	Publicação no Docker Hub com as tags latest e SHA do commit.
CD do backend	Deploy hook do Render executado depois do sucesso das duas CIs.
CD do frontend	Vercel pull, build de produção e deploy prebuilt.
Segurança	Permissão contents: read, Secrets, Variables, environments e validação das credenciais.
Controle	Timeouts e cancelamento de execuções antigas da mesma referência.
Documentação	README atualizado com execução Docker e configuração do CI/CD.
Versionamento	Commits 22be71e, e58c01d e 9563b7b publicados na branch codex/implementa-lab-cicd.

5. Breve análise do processo

A IA foi utilizada para interpretar os requisitos, estruturar os jobs e revisar o workflow. As sugestões precisaram ser adaptadas à arquitetura real do projeto e às regras do laboratório. A principal decisão foi separar CI e CD: pull requests validam o código sem realizar deployment, enquanto pushes na main habilitam o CD somente depois que frontend e backend forem aprovados. O uso do SHA como tag da imagem permite relacionar a versão publicada ao commit validado.

6. Evidências da execução correta da pipeline CI/CD

Evidências ainda não disponíveis

O novo workflow ainda não possui uma execução concluída no GitHub Actions. Portanto, não é possível inserir evidências reais da execução correta sem configurar as credenciais, abrir o Pull Request e executar o fluxo na branch main. Nenhuma evidência fictícia foi adicionada.